EcoLoop

Circular material systems and zero-cost digital architecture.

Swami Sant Dass Public School, Jalandhar

The Packaging Trap: Multi-Layer Plastics

Why standard municipal recyclers throw clean packaging straight into landfills.

The Material Reality

- **Bonded Metal & Plastic:**
  Chips and snack packets laminate aluminum foil between plastic film. Separating those layers is chemically difficult and commercially unprofitable.
- **100% Landfill Destination:**
  Because municipal plants cannot process multi-layer packaging, almost all snack wrappers are burned or dumped into local landfills.
- **Clean Waste in Schools:**
  Canteens and classrooms generate sacks of clean snack wrappers daily with zero realistic collection logistics.

The EcoLoop Approach

- **Skip Separation, Heat-Compress:**
  Instead of separating the foil, we partner to heat-press whole wrappers into dense boards that replace plywood.
- **Unified Circular Model:**
  Unifying scattered school green crafts and outsourced heavy materials into one clean system.
- **Digital Transparency:**
  Building an online catalog so buyers can see exact specifications and request real samples.

The Physical Loop

Six circular material systems systematically diverted from school waste streams.

STUDENT-MADE

Seed Paper

Handmade plantable paper embedded with native seeds, crafted directly inside school.

STUDENT-MADE

Bioenzymes

Citrus fruit peels fermented into natural multi-purpose cleaning liquids for campus sanitation.

STUDENT-MADE

Uniform Bags

Durable, reusable tote bags stitched from discarded school uniform fabrics.

STUDENT-MADE

Eco-Bricks

PET bottles tightly packed with clean plastic wrappers for modular walls and simple structures.

STUDENT-MADE

Bottle Planters

Upcycled beverage containers for school corridor nursery and vertical greenery installations.

BULK EXCHANGE

MLP Sheets

Heavy-duty structural boards heat-pressed by an external NGO from bulk wrapper batches.

how it started vs how it's going

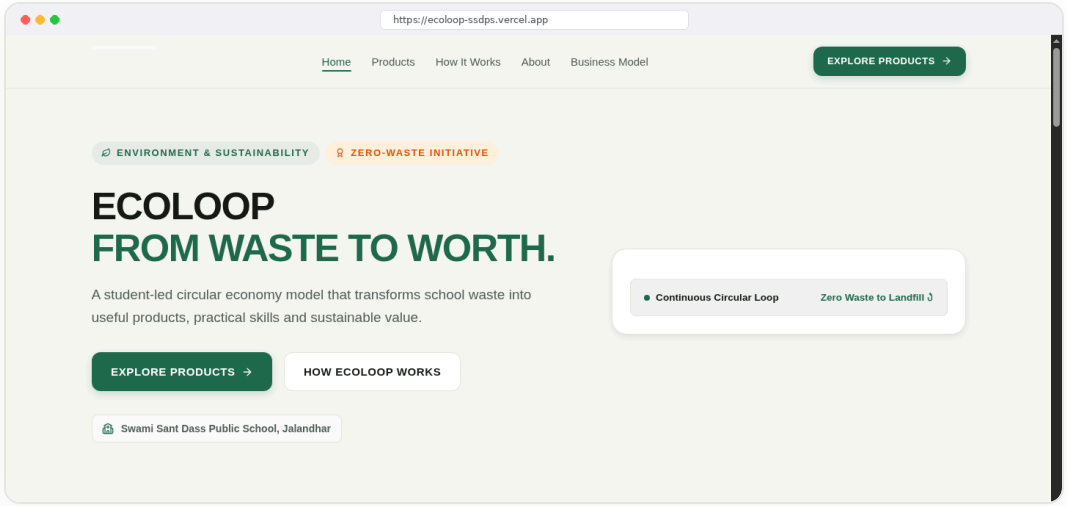
student crafts and the wrapper exchange were already happening. i built the circular reinvestment loop and the digital storefront.

Why Build a Digital Platform?

Transforming scattered school crafts into a unified, public circular initiative.

Platform Objectives

- **Unified Public Presence:**
  Showcasing all circular materials and workflows in one public place so it operates as a serious initiative, not scattered classroom crafts.
- **Direct Inquiry Routing:**
  Giving outside people and institutions a friction-free way to contact us and request physical samples.
- **Connecting School to Outside World:**
  Taking activities happening inside classrooms and presenting them professionally to external organizations.



The Zero-Cost Backend: Built on Google Sheets

Handling live incoming inquiries without paid servers or corporate credit cards.

STEP 01

React Form

User fills in their name, product choice, and notes. The browser validates all inputs locally.

→

STEP 02

Apps Script Webhook

A free Google Apps Script webhook receives the data payload securely in the cloud.

→

STEP 03

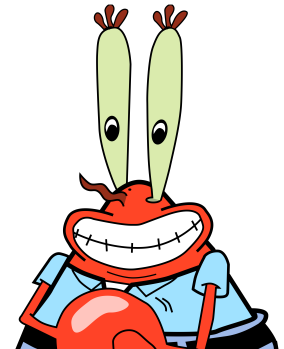
Sheets Database

Appends the inquiry row into a live Google Sheet and immediately triggers an email alert.

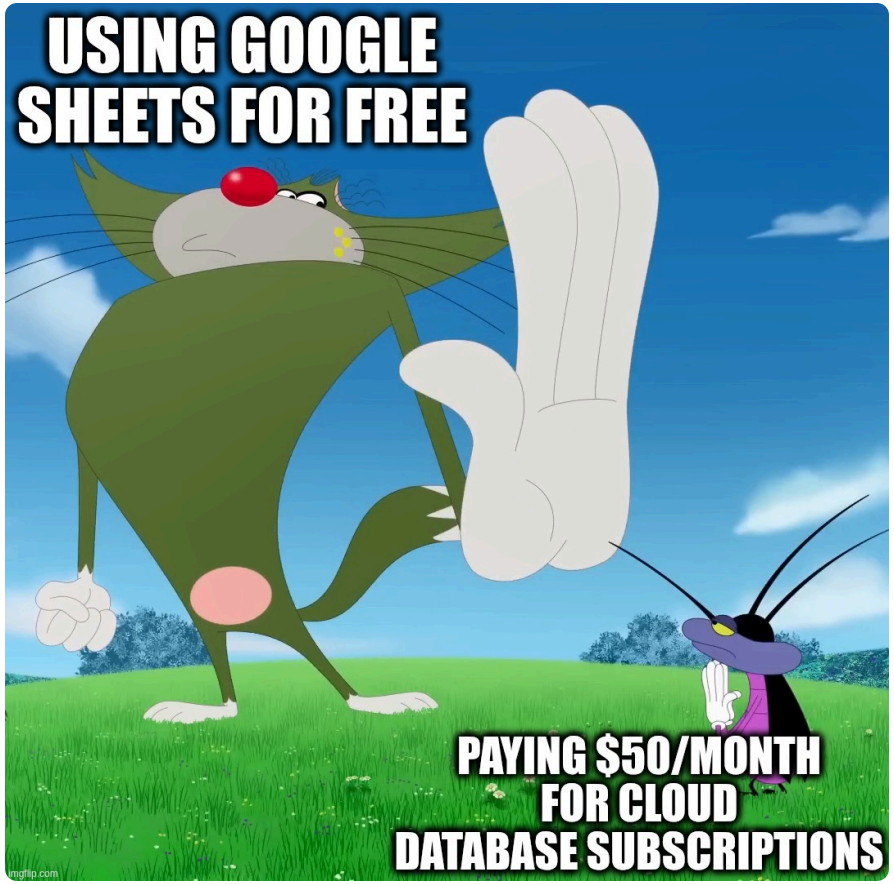
Monthly Infrastructure Bill

No credit cards, zero server maintenance, and reliable for real order leads.

$0.00 / month



why our database bill is literally $0



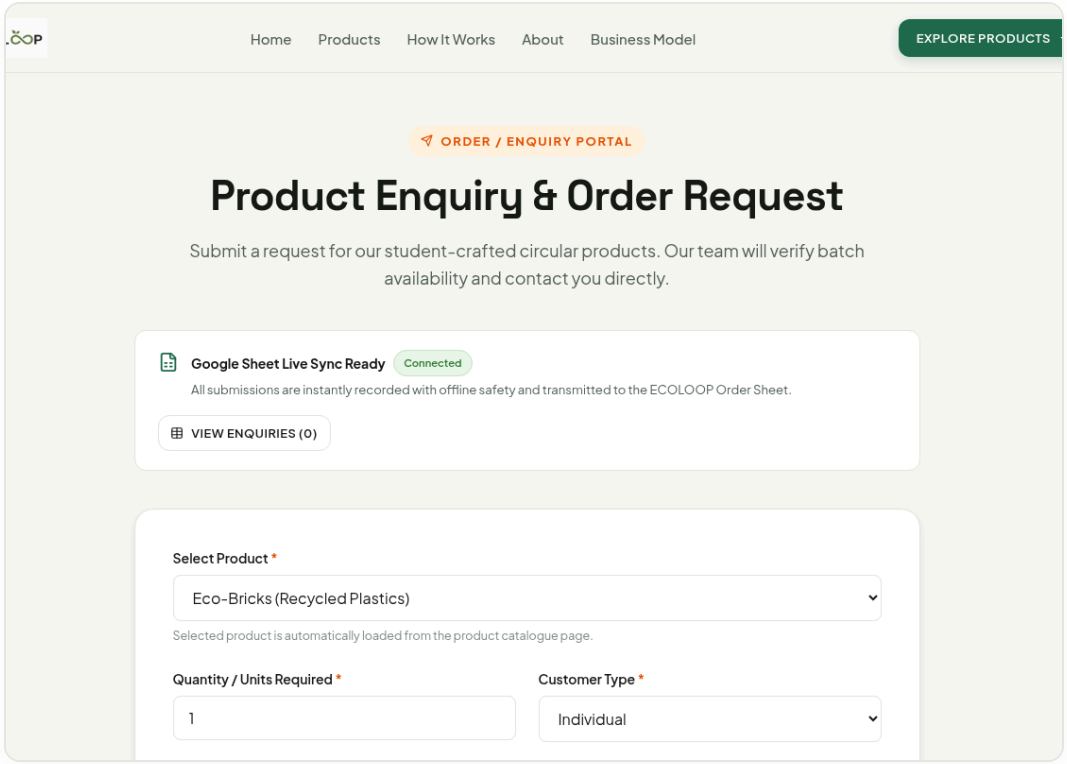
why spend monthly money on complex database servers when a free google sheet captures all your incoming inquiries?

The Inquiry Engine: An Honest Review

Built in a single night sprint: functional, basic, and with plenty of room to improve.

The Reality of Our MVP

- **Built in One Night:**
  Put together in a single late-night sprint to give the exhibition judges a working demo to test.
- **Basic but Working:**
  Automatically pre-populates product details and logs records straight into our backend without paying for services.
- **Room for Improvement:**
  Needs automated inventory deduction, better error states, and order tracking. It's an MVP, not final software.



How I Built It Without Knowing React

Moving from typing boilerplate code to designing systems with AI agents.

The Traditional Hurdle

- Getting stuck on missing brackets, package version mismatches, and unfamiliar framework setup.
- Spending entire evenings digging through Stack Overflow threads instead of building real features.
- Giving up on software ideas simply because you haven't spent 6 months memorizing syntax.

Developer as Architect

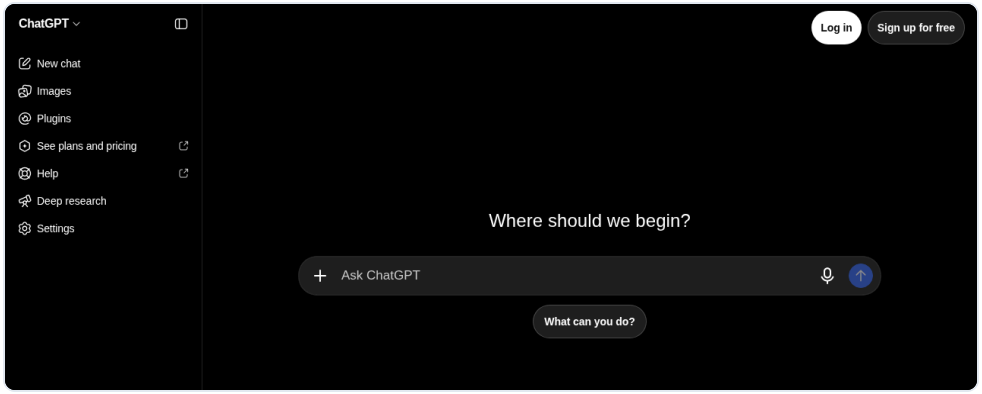
- You specify the data models, visual style, and constraints in plain language.
- An autonomous agent creates components, connects routes, and modifies multiple files.
- Your primary job becomes quality control, testing user flows, and refining the architecture.

Chatbot vs Agent Workspace

The difference between copy-pasting snippets and letting an agent build inside your files.

ChatGPT: Web Chatbot

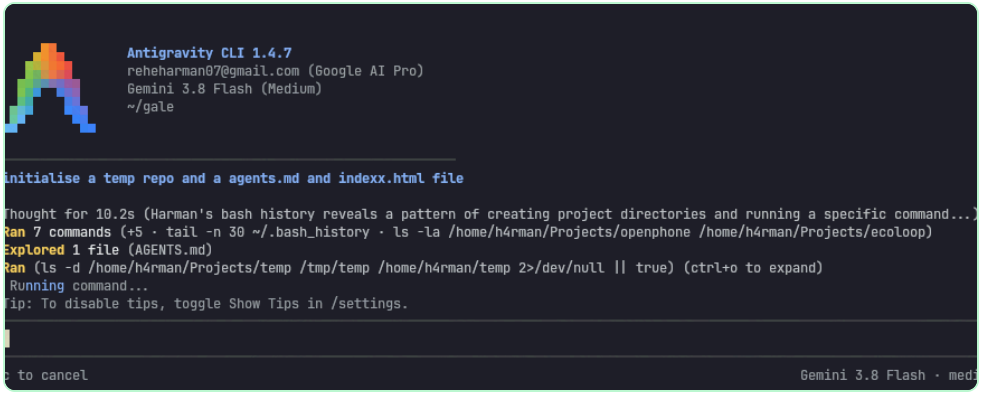
ISOLATED



- **Copy-Paste Loop:** Generates isolated code snippets; you must manually paste and fix them.
- **Zero File Access:** Cannot see your directory tree, configs, or local dependencies.
- **Blind Execution:** Cannot run builds, view terminal errors, or verify live changes.

Antigravity: Agent Workspace

AUTONOMOUS



- **Direct Repository Edits:** Reads folder context and writes multi-file modifications directly.
- **Autonomous Terminal:** Executes shell commands, dev servers, and build scripts.
- **Self-Repair Loop:** Catches compiler and runtime errors and fixes them automatically.

How Working with an Agent Actually Looks

A practical 4-step loop of specifying, editing, testing, and self-repairing.

STAGE 01

Write the Spec

Describe what you want in plain language: colors, component layout, and behavior rules.

STAGE 02

Agent Edits Files

The agent inspects your repository, writes components, and updates project files directly.

STAGE 03

You Test Locally

You run the app in your browser, click the buttons, and see what works or what broke.

STAGE 04

Agent Fixes Bugs

You paste the error back to the agent; it reads the log and repairs the broken code.

Replicating This Workflow: OpenCode

An open-source terminal agent that gives you the same autonomous capabilities.

Why OpenCode for Students?

- **100% Free & Open Source:**
  Created by Anomaly (GitHub: `anomalyco/opencode`). Zero subscriptions or vendor lock-in.
- **Lives in Your Terminal:**
  Reads your project files, edits code, and runs local commands just like specialized agent harnesses.
- **Dual Execution Modes:**
  Plan Mode reviews architecture safely; Build Mode executes real code changes.

```
// Quick Installation in 60 Seconds

$ npm install -g @opencode/cli
$ cd ./my-project
$ opencode

Runs on macOS, Linux, and Windows WSL.
```

Free AI Models: Zero Cost for Students

Practical options you can use tonight without paying for subscriptions.

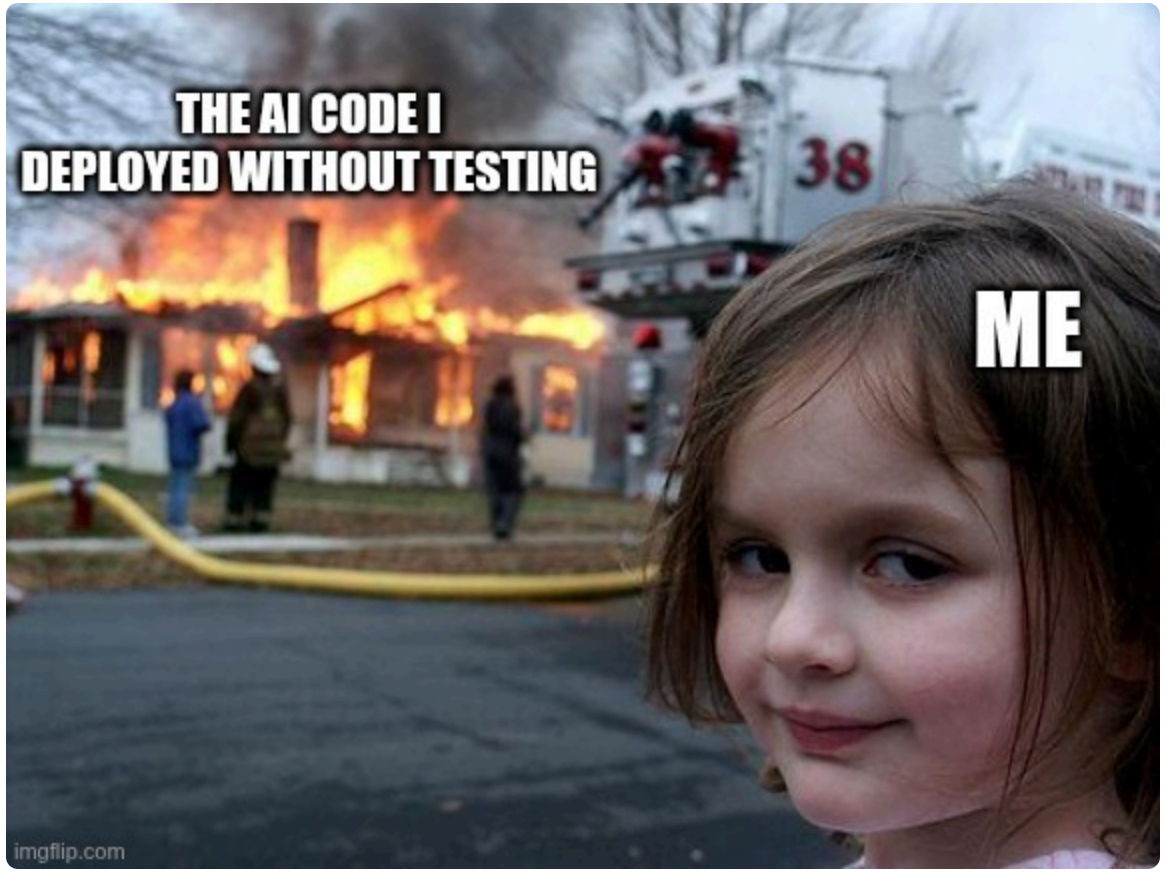
Free Cloud APIs

- **Google Gemini API:**
  Sign in to `aistudio.google.com` for a free key. Generous free limits for fast coding models.
- **Groq Cloud:**
  Ultra-fast free inference on open models like Llama 3.3 via `console.groq.com`.
- **GitHub Models:**
  Free student access to preview models inside VS Code and GitHub Codespaces.

100% Offline with Ollama

- **Private & Local:**
  Download `ollama.com`. Runs entirely on your laptop hardware without needing internet.
- **Recommended Model:**
  `qwen2.5-coder:7b` delivers strong coding reasoning on standard student laptops.
- **Zero Latency:**
  Fast responses directly on your machine with complete data privacy.

deployed without testing



prompting is only 20% of the work. clicking every button to see what actually broke is the other 80%.

Thank You.

Scan the code to access the complete open-source repository, live demo, and student setup guide.

`github.com/Harman209/ecoloop`

Swami Sant Dass Public School, Jalandhar